

Analytics Engineering: Session 18

Practice Session V: Incremental Models & Snapshots

Goal

Build an incremental model and a snapshot from scratch, then observe how dbt behaves across different run scenarios.

By the end of this session you will have:

1. `mart_daily_sessions` — an incremental model aggregating website traffic by day
2. Tests for `mart_daily_sessions` passing on first run, second run, and `--full-refresh`
3. `customer_snapshot` — a check-strategy snapshot tracking attribute changes
4. Live proof that dbt captured an attribute change as a new SCD2 row

Checklist

- [] `models/marts/mart_daily_sessions.sql` — incremental, `unique_key: session_day`, `on_schema_change: append_new_columns`
- [] `models/marts/_mart_daily_sessions.yml` — `session_day` unique + not_null; all other columns not_null
- [] `dbt build -s mart_daily_sessions` passes on **first run**
- [] `dbt build -s mart_daily_sessions` passes on **second run** (incremental path)
- [] `dbt build -s mart_daily_sessions --full-refresh` passes
- [] `snapshots/customer_snapshot.sql` — `check` strategy, watching `customer_segment` and `country`
- [] `dbt snapshot` then mutate a customer then `dbt snapshot` again — two rows for the mutated customer

The DAG



mart_daily_sessions aggregates **stg_website_sessions** by day. On subsequent runs it reads the max date from **{{ this }}** and only processes new rows.

Exercise 1— mart_daily_sessions (30 min)

Create `models/marts/mart_daily_sessions.sql` using `{{ ref('stg_website_sessions') }}`.

Requirements:

- `materialized='incremental'`, `unique_key='session_day'`,
`on_schema_change='append_new_columns'`
- The incremental filter must use `{% if is_incremental() %}` to filter rows where the session date is **greater than or equal to** the max `session_day` already in `{{ this }}`
- Source date columns are `VARCHAR` — cast before comparing

Exercise 1 — output columns

Column	Type	Description
<code>session_day</code>	<code>date</code>	<code>session_date</code> cast to <code>date</code> — the grain of the model
<code>sessions</code>	<code>integer</code>	Total sessions for that day
<code>conversions</code>	<code>integer</code>	Sessions where <code>converted = true</code>
<code>conversion_rate</code>	<code>numeric</code>	<code>conversions / sessions * 100</code> , rounded to 2 d.p.
<code>unique_sources</code>	<code>integer</code>	Distinct values of <code>source</code> for that day

Exercise 1 — tests

Create `models/marts/_mart_daily_sessions.yml` .

Add generic tests:

- `session_day` : `unique` , `not_null`
- `sessions` , `conversions` , `conversion_rate` , `unique_sources` : `not_null`

Exercise 2 — Run scenarios (25 min)

① **First run** — full build, no WHERE filter yet:

```
dbt build -s mart_daily_sessions
```

Query the mart: how many days are there?

② **Add new sessions** — grow the source table:

```
uv run python add_sessions.py    # ~120 new sessions across 3 new days
```

Query staging **without running dbt** — do the new rows appear?

```
select max(cast(session_date as date)), count(*) from main.stg_website_sessions;
```

Exercise 2 — Run scenarios (cont.)

③ **Incremental run** — only the new days should be processed:

```
dbt build -s mart_daily_sessions
```

Query the mart again: did it grow by exactly 3 days?

Inspect the compiled SQL — the **WHERE** predicate should now be visible:

```
dbt compile -s mart_daily_sessions  
# target/compiled/dbt_ie/models/marts/mart_daily_sessions.sql
```

④ **Force rebuild** — drop and recreate from all source data:

```
dbt build -s mart_daily_sessions --full-refresh
```

Compile again — is the **WHERE** predicate gone?

Exercise 3 — customer_snapshot (20 min)

Create `snapshots/customer_snapshot.sql`.

Requirements:

- `target_schema='snapshots'`
- `unique_key='customer_id'`
- `strategy='check'`
- `check_cols=['customer_segment', 'country']`
- Select all columns from `{{ source('raw', 'customers') }}`

Run it:

```
dbt snapshot
```

Confirm the snapshot has **one row per customer** and that the `dbt_valid_to` column is

`11` for all rows

Exercise 3 — capture a change

Open a DuckDB session and mutate one customer:

```
update main.customers
  set customer_segment = 'enterprise'
where customer_id = 1;
```

Run the snapshot again:

```
dbt snapshot
```

Then query the result:

```
select customer_id, customer_segment, dbt_valid_from, dbt_valid_to
  from snapshots.customer_snapshot
where customer_id = 1
order by dbt_valid_from;
```

Verify

```
# Reset to a clean state before starting  
python create_db.py
```

```
# Exercise 2 – four-step incremental demo
```

```
dbt build -s mart_daily_sessions           # ① first run  
uv run python add_sessions.py             # ② inject 3 new days  
dbt build -s mart_daily_sessions           # ③ incremental – only new days processed  
dbt build -s mart_daily_sessions --full-refresh # ④ force rebuild
```

```
# Exercise 3 – snapshot
```

```
dbt snapshot                               # initial capture  
# (mutate main.customers in DuckDB)  
dbt snapshot                               # capture the change
```

Bonus

- Change `on_schema_change` to `'fail'`, add a new column to `mart_daily_sessions`, and run without `--full-refresh` — read the error message
- Add `avg_duration_seconds` (average of `session_duration_seconds`) to `mart_daily_sessions` — handle the schema change correctly
- Change `check_cols` to `'all'` in `customer_snapshot`, then update a column **not** in the original `check_cols` list (e.g., `first_name`) — does dbt create a new row?
- Change the incremental strategy from `merge` to `delete+insert` — compare the compiled SQL

What have we practiced in this session

- **Incremental models:** `is_incremental()` filter, `unique_key`, `merge` strategy, casting VARCHAR dates, `on_schema_change`
- **Run scenarios:** first run vs subsequent run vs `--full-refresh` — observed in compiled SQL
- **Snapshots:** `check` strategy, `dbt_valid_from` / `dbt_valid_to`, capturing a live attribute change as SCD2